Code-Layout, Readability and Reusability

Date	15 July 2026
Team ID	xxxxxx
Project Name	**StudyAI – Smart AI educate Companion & Quiz Generator**
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

Code Quality & Structure Details

S.No	Quality Principle	Implementation Details / Description	Specific Project Examples (Files/Folders)
1	**Folder/Directory Structure**	*The project follows a component-based and layered architecture, separating routes, components, integrations, and utility functions for better maintainability.*	*/src/routes, /src/components, /src/lib, /src/integrations, /supabase*
2	**Naming Conventions**	*Variables and functions use **camelCase**, React components use **PascalCase**, and file names are descriptive and meaningful.*	*FlashcardsPage.tsx, generateQuizFromText(), DocPicker.tsx, ai.functions.ts*
3	**Code Readability & Comments**	*Code is organized into small functions with meaningful names. Comments are added to explain complex logic and API integrations when necessary.*	*quizGenerator.ts, flashcards.tsx, ai.functions.ts*
4	**Modularity & Reusability**	*Reusable UI components and utility functions are created to avoid code duplication and improve maintainability (DRY principle).*	*DocPicker.tsx, reusable button components, utility functions in /src/lib*

S.No	Quality Principle	Implementation Details / Description	Specific Project Examples (Files/Folders)
5	**Formatting & Linting Tools**	*The project uses automated formatting and linting tools to maintain consistent coding standards.*	*Prettier, ESLint, .prettierrc, eslint.config.js, package.json*
6	**Error Handling Patterns**	*Errors are handled using try-catch blocks, toast notifications, and proper API response validation to provide meaningful feedback to users.*	*flashcards.tsx, quizGenerator.ts, Supabase API calls, AI API integrations*